CS336 作业 4（数据）

语言模型数据过滤

中文翻译版

版本 26.0.1

CS336 课程组

2026 年春季

注：代码、命令、路径、函数名、测试名及文献条目保留原始形式，以避免技术歧义。

1 作业概览

在本次作业中，你将通过亲手处理网页抓取数据，获得有关“如何过滤网络爬取结果并构建语言模型训练数据”的实践经验。

你将实现的内容

1. 将 Common Crawl 的 HTML 转换为文本。
2. 使用多种方法过滤抽取出的文本，例如过滤有害内容、处理个人可识别信息等。
3. 对训练数据进行去重。

你将运行的实验

4. 在不同数据集上训练语言模型，以更好地理解具体数据处理决策对模型性能的影响。

代码结构

本次作业的全部代码以及本说明文档均位于以下 GitHub 仓库：

```
github.com/stanford-cs336/assignment4-data
```

请使用 git clone 克隆该仓库。如有更新，课程组会通知你，届时可使用 git pull 获取最新版本。

1. cs336_basics/*：该目录包含用于训练作业 1 中所构建模型的代码。模型经过了少量优化，将作业 1 中部分手工实现的组件替换为 PyTorch 原生实现，例如使用 PyTorch 内置的交叉熵计算内核。课程组还提供了支持多 GPU 分布式数据并行训练的脚本。你将使用该脚本在自己过滤后的数据上训练模型。

2. cs336_data/*：你将在这里编写作业 4 的代码。课程组已经创建了名为 cs336_data 的模块，其中包含起始代码和需要补全的占位内容。

3. tests/*.py：该目录包含你必须通过的全部测试。这些测试会调用 tests/adapters.py 中定义的接口钩子。你需要实现适配器，将自己的代码连接到测试。为了调试，你可以编写更多测试或修改测试代码，但最终实现必须能够通过课程组原始提供的测试套件。

4. README.md：该文件提供预期目录结构的更多细节，以及环境配置的基本说明。

提交方式

你需要向 Gradescope 提交以下文件：

- writeup.pdf：回答所有书面问题。请使用排版工具整理你的答案。
- code.zip：包含你编写的全部代码。

2 过滤 Common Crawl

虽然大型语言模型主要使用互联网数据进行训练，但大多数研究人员不会自行构建网络爬虫来获取模型训练数据，而是使用公开可用的网页抓取结果。其中最常用的公开网络抓取数据来自 Common Crawl。这是一家非营利机构，免费提供网页语料库，覆盖“17 年间超过 2500 亿个网页”。[1]

不过，正如后文将看到的那样，将 Common Crawl（CC）的数据转储加工成可供语言模型训练的数据，需要大量工作。例如，网页原始内容是 HTML，而我们希望从中抽取文本。此外，许多网页可能质量较

低、完全重复或近似重复，可能含有有害内容或敏感信息。我们可能需要删除这些网页，或者从数据集中移除其中不希望保留的部分。本次作业将搭建一条执行若干上述步骤的数据处理流水线，把原始互联网数据转化为可用于语言模型训练的数据集。

2.1 查看数据

在开始实现任何功能之前，先查看原始数据并建立直观认识通常很有帮助。Common Crawl 数据提供三种格式：

WARC

WARC（Web ARChive format，网页归档格式）文件保存原始 CC 数据，包括网页 ID 和 URL、元数据与 HTTP 请求细节（例如请求日期和时间、服务器 IP 地址），以及网页的原始内容（例如 HTML）。

WAT

WAT（Web Archive Transformation，网页归档转换）文件保存从 WARC 文件中抽取出的高层元数据，并以 JSON 对象形式写出。以 HTML 网页为例，其中包括网页中的链接列表和网页标题。

WET

WET（Web Extracted Text，网页抽取文本）文件保存从原始 HTML 网页中抽取出的纯文本。

在接下来的题目中，我们将查看一个 WARC 文件及其对应的 WET 文件。这些文件来自 2026 年的一次 Common Crawl 抓取，下载命令如下：

```
# 下载一个示例 WARC 文件。
$ wget https://data.commoncrawl.org/crawl-data/CC-MAIN-2026-12/segments/1741613651012.6/warc/CC-MAIN-20260310152245-20260310182245-00000.warc.gz
# 下载其对应的 WET 文件。
$ wget https://data.commoncrawl.org/crawl-data/CC-MAIN-2026-12/segments/1741613651012.6/wet/CC-MAIN-20260310152245-20260310182245-00000.warc.wet.gz
```

这些文件位于：

- /shared-data/CC/example.warc.gz
- /shared-data/CC/example.warc.wet.gz

警告：这些文件包含完全未经筛选的互联网网页，其中可能存在大量潜在有害内容。若看到不愿阅读的文档，可以继续向下滚动并跳过它。

[1] https://commoncrawl.org/

题目（look_at_cc）：查看 Common Crawl（4 分）

(a) 使用上文路径中提供的 WARC 文件副本。先查看该文件中的第一个网页。该文件经过 gzip 压缩，可以使用以下命令浏览其内容：

```
$ zcat /shared-data/CC/example.warc.gz | less
```

less 允许你使用键盘方向键、Page Up 和 Page Down 浏览文件。按“q”退出。

查看第一个网页：它的 URL 是什么？现在是否仍可访问？仅查看原始 HTML，你能判断该网页大致在讲什么吗？

交付内容：用 2-3 句话回答。

(b) 接下来查看对应的 WET 文件：

```
$ zcat /shared-data/CC/example.warc.wet.gz | less
```

注意，WET 文件中包含 HTTP 头部，例如 Content-Length；这些头部并不属于抽取出的正文内容。查看第一个样例时，你会看到从刚才的原始 HTML 中抽取出的文本。

你会发现，抽取文本中的相当一部分更像是 HTML 页面结构的残留，而不是真正的网页主体内容。哪些文本片段本应被抽取器过滤掉？从训练数据质量角度考虑，在这类文本上训练模型可能会出现什么问题？反过来，模型可能从该网页中学到哪些有用信息？

交付内容：用 3-4 句话回答。

(c) “好的训练样例”高度依赖具体场景。请描述一个适合将该样例保留在训练数据中的应用领域，以及一个不适合保留该样例的应用领域。

交付内容：用 1-2 句话回答。

(d) 再查看一些样例，以更好地理解 Common Crawl 中包含的内容。继续浏览 25 条 WET 记录。对每条记录，极简地记录文档语言（若能识别）、域名、网页类型等信息。你查看到第几个样例时，才第一次遇到自己认为“高质量”的网页？

交付内容：对 25 个文档分别给出简短标注，包括文档语言、域名、网页类型及其他备注；同时报告看到第几个样例时出现第一个高质量网页。

2.2 HTML 转文本

在查看上述 WARC 和 WET 文件后，你应该已经意识到：从 HTML 中抽取文本并不容易。通常，文本抽取流程会寻找 HTML 中的可见内容，例如本应包含文本段落的 <p> 标签。但即使如此，抽取结果仍可能远多于用户在浏览器中看到并认为是网页主体的内容。例如，打开 StackOverflow 时，主体内容是问题与回答；但从技术上说，菜单选项、指向其他 StackExchange 站点中无关页面的链接、页脚、注册或登录链接等，也都是可见文本。可靠地区分这些内容与网页主体非常困难。

许多工具都实现了文本抽取流水线。本次作业使用 Resiliparse[2] 库执行文本抽取。Resiliparse 还可以帮助解决一个更基础的问题：检测承载原始网页内容的字节序列所采用的文本编码。虽然根据 Wikipedia 的统计，网页中有 98.2% 使用 UTF-8 编码，但我们的文本抽取流水线也应当能够稳健处理其他编码。

说明：建议使用 FastWARC 库遍历每个 WARC 文件中的记录。下列类可能会有帮助：

```
from fastwarc.warc import ArchiveIterator, WarcRecordType
```

题目（extract_text）：HTML 转文本（3 分）

(a) 编写一个函数，从包含原始 HTML 的字节串中抽取文本。使用 resiliparse.extract.html2text.extract_plain_text 执行抽取。该函数要求输入字符串，因此你需要先将字节串解码为 Unicode 字符串。注意，输入字节串不一定采用 UTF-8 编码；若 UTF-8 解码失败，你的函数应能检测其实际编码。Resiliparse 还提供 resiliparse.parse.encoding.detect_encoding()，可能会有帮助。

交付内容：实现一个函数：输入包含 HTML 的字节串，返回抽取出的文本字符串。实现适配器 [run_extract_text_from_html_bytes]，并确保通过 uv run pytest -k test_extract_text_from_html_bytes。

(b) 在一个 WARC 文件上运行你的文本抽取函数，并将输出与对应 WET 文件中的抽取文本进行比较。你观察到了哪些相同点和不同点？哪一种抽取结果更好？

交付内容：用 2-3 句话比较你自己的函数与 WET 文件中的抽取结果。

[2] https://resiliparse.chatnoir.eu/en/stable/index.html

2.3 语言识别

互联网中包含用数千种语言编写的网页。然而，在大多数算力预算下，大规模训练一个能够有效利用如此多样数据的多语言模型都非常困难。因此，许多从 Common Crawl 构建的语言模型训练集只保留有限种语言的数据。

fastText（https://fasttext.cc）是实现该功能的一个实用库，它提供高效的文本分类器。该库既提供训练自定义分类器的基础设施，也提供一系列预训练模型，其中包括语言识别模型。你可以从 https://fasttext.cc/docs/en/language-identification.html 下载 fastText 语言识别模型 lid.176.bin；该模型也位于 /shared-data/classifiers/lid.176.bin。

语言过滤器通常根据分类器给出的分数决定是否保留某个网页。请使用 fastText 语言识别分类器实现一个语言识别过滤器，并让它输出一个非负分数，用于表示分类器对预测结果的置信程度。

题目（language_identification）：语言识别（6 分）

(a) 编写一个函数，输入 Unicode 字符串，识别其中主要使用的语言。函数应返回一个二元组，其中包含语言标识符，以及一个 0 到 1 之间的分数，用于表示该预测的置信度。

交付内容：实现语言识别函数，返回置信度最高的语言预测及其分数。实现适配器 [run_identify_language]，并确保通过 uv run pytest -k test_identify_language 中的两个测试。注意，测试假定英语标识符为“en”、中文标识符为“zh”；如有必要，你的测试适配器应执行相应的重映射。

(b) 语言模型在推理阶段的行为很大程度上取决于训练数据。因此，数据过滤流水线中的问题可能引发下游问题。语言识别过程出错可能造成哪些问题？在更高风险的场景中，例如部署面向用户的产品，你会如何缓解这些问题？

交付内容：用 2-5 句话回答。

(c) 在从 WARC 文件抽取出的文本上运行你的语言识别系统，抽取过程使用之前实现的文本抽取函数。随机选择 20 个样例，手工判断其语言，并与分类器预测进行比较。报告分类器出现的错误。文档中有多大比例是英语？根据观察，你认为过滤时应采用怎样的分类器置信度阈值？

交付内容：用 2-5 句话回答。

2.4 个人可识别信息

互联网中包含大量可以联系或识别个人的信息，例如电子邮件地址、电话号码和 IP 地址。我们可能不希望面向用户的语言模型输出真实个人的这类信息，因此常见的数据处理步骤是对训练数据中的这些信息进行遮蔽。

接下来，你将实现三个遮蔽过程，分别处理：（a）电子邮件地址；（b）电话号码；（c）IP 地址。

题目（mask_pii）：个人可识别信息（3 分）

(a) 编写一个遮蔽电子邮件地址的函数。函数输入一个字符串，将其中所有电子邮件地址替换为字符串 "|||EMAIL_ADDRESS|||"。你可以查找能够较可靠识别电子邮件地址的正则表达式。

交付内容：实现一个函数，将给定字符串中的所有电子邮件地址替换为 "|||EMAIL_ADDRESS|||"，并返回一个二元组，其中包含新字符串和被遮蔽的实例数量。实现适配器 [run_mask_emails]，并确保通过 uv run pytest -k test_mask_emails 的全部测试。

(b) 编写一个遮蔽电话号码的函数。函数输入一个字符串，将其中所有电话号码替换为字符串 "|||PHONE_NUMBER|||"。可靠识别电话号码极其困难，因为电话号码可能采用非常多样的格式；但你至少应覆盖美国最常见的电话号码格式，并对轻微的语法变化保持稳健。

交付内容：实现一个函数，将给定字符串中的电话号码替换为 "|||PHONE_NUMBER|||"，并返回一个二元组，其中包含新字符串和被遮蔽的实例数量。实现适配器 [run_mask_phone_numbers]，并确保通过 uv run pytest -k test_mask_phones。

(c) 编写一个遮蔽 IP 地址的函数。本题只需要处理 IPv4 地址，即由点分隔的 4 个数字，每个数字最大为 255。函数输入一个字符串，将其中所有 IP 地址替换为字符串 "|||IP_ADDRESS|||"。

交付内容：实现一个函数，将给定字符串中的 IPv4 地址替换为 "|||IP_ADDRESS|||"，并返回一个二元组，其中包含新字符串和被遮蔽的实例数量。实现适配器 [run_mask_ips]，并确保通过 uv run pytest -k test_mask_ips。

(d) 如果在训练集上朴素地应用这些过滤器，你认为语言模型在下游可能出现哪些问题？如何缓解这些问题？

交付内容：用 2-5 句话回答。

(e) 在从 WARC 文件抽取出的文本上运行你的 PII 遮蔽函数。随机查看 20 个发生替换的样例，并给出一些误报与漏报示例。

交付内容：用 2-5 句话回答。

2.5 有害内容

未经筛选的互联网数据转储中包含大量不希望语言模型在推理时复述的文本。令人意外的是，其中一些训练样例甚至来自通常被认为无害的网站，例如 Wikipedia：若干页面下的用户评论可能非常有毒。尽管几乎不可能为“有害”划定一条完全清晰的边界，许多数据过滤流水线仍会过滤那些主要包含有害内容的网页。

识别此类内容的方法很多，包括统计禁用词表中的词语，或使用人工标注的数据训练简单分类器。本部分聚焦两类广义有害内容：“不适合工作场所”（Not Safe For Work，NSFW）内容，其中包括色情、脏话或其他潜在令人不适的内容；以及有毒言论，即“粗鲁、不尊重或不讲道理，且很可能让人退出讨论的语言”。[3] 我们将使用 Dolma 项目[1]提供的 fastText 预训练模型，判断输入文本是否属于上述类别。这些分类器使用 Jigsaw Toxic Comments 数据集训练，该数据集包含按多种标签分类的 Wikipedia 评论。[4]

NSFW 分类器可从以下地址下载：

```
dolma-artifacts.org/.../jigsaw_fasttext_bigrams_nsfw_final.bin
```

仇恨言论分类器可从以下地址下载：

```
dolma-artifacts.org/.../jigsaw_fasttext_bigrams_hatespeech_final.bin
```

课程组也已将这两个分类器放入共享数据目录：

- /shared-data/classifiers/dolma_fasttext_hatespeech_jigsaw_model.bin：用于检测仇恨与有毒言论的预训练分类器。
- /shared-data/classifiers/dolma_fasttext_nsfw_jigsaw_model.bin：用于检测 NSFW 内容的预训练分类器。

使用这些模型实现一个函数：输入包含网页内容的 Unicode 字符串，返回标签（例如“toxic”“non-toxic”）及其置信度分数。

题目（harmful_content）：有害内容（6 分）

(a) 编写一个检测 NSFW 内容的函数。

> **交付内容：**实现一个函数，判断给定字符串是否包含 NSFW 内容，并返回标签与置信度分数构成的二元组。实现适配器 [run_classify_nsfw]，并确保通过 uv run pytest -k test_classify_nsfw。注意，该测试只是取自 Jigsaw 数据集的基本健全性检查，并不意味着分类器足够准确；你仍应自行验证其表现。

(b) 编写一个检测有毒言论的函数。

> **交付内容：**实现一个函数，判断给定字符串是否由有毒言论构成，并返回标签与置信度分数构成的二元组。实现适配器 [run_classify_toxic_speech]，并确保通过 uv run pytest -k test_classify_toxic_speech。该测试同样只是取自 Jigsaw 的基本健全性检查。

(c) 将这些过滤器用于构建训练集时，语言模型在下游可能出现哪些问题？如何缓解这些问题？

交付内容：用 2-5 句话回答。

(d) 在从 WARC 文件抽取出的文本上运行有害内容过滤器。随机查看 20 个样例，将分类器预测与你自己的判断进行比较，并报告分类器错误。有多大比例的文档属于有害内容？根据观察，过滤时应采用怎样的分类器置信度阈值？

交付内容：用 2-5 句话回答。

[3] https://current.withgoogle.com/the-current/toxicity/

[4] https://paperswithcode.com/dataset/toxic-comment-classification-challenge

2.6 质量规则

即使已经按语言过滤网页并移除有害内容，剩余网页中仍会有相当一部分不适合用于语言模型训练。虽然“质量”同样很难精确定义，但查看 Common Crawl 样例有助于识别一些典型的低质量内容，例如：

- 带付费墙的内容。
- 失效链接对应的占位页面。
- 登录、注册或联系表单。
- 主要由非文本内容构成、而这些内容在文本抽取时会丢失的网页，例如照片或视频页面。

Gopher 论文[2]描述了一组简单的质量过滤器，用于从网页抓取数据中移除类似的、显而易见的低质量文本。这些过滤器由易于理解的启发式规则构成，通常能够覆盖大量明显不适合作为训练样例的情况。Gopher 质量过滤器包含基于文档长度、词长、符号与词的比例，以及特定英语停用词是否出现等标准。

本次作业要求实现 Gopher 论文中一部分过滤规则。具体而言，应移除满足下列任一条件的文档：

- 词数少于 50 或多于 100,000。
- 平均词长不在 3 到 10 个字符之间。
- 超过 30% 的行以省略号“...”结尾。
- 至少包含一个字母字符的词少于全部词的 80%。

关于 Gopher 论文使用的全部质量过滤器，请参阅其附录 A。

题目（gopher_quality_filters）：Gopher 质量过滤器（3 分）

(a) 至少实现上述 Gopher 质量过滤器子集。对文本进行词语切分时，可以使用 NLTK 包，尤其是 nltk.word_tokenize；但并不强制使用。

交付内容：实现一个函数，仅接收一个字符串参数，并返回布尔值，表示文本是否通过 Gopher 质量过滤器。实现适配器 [run_gopher_quality_filter]，并确保通过 uv run pytest -k test_gopher 中的测试。

(b) 在从 WARC 文件抽取出的文本上运行基于规则的质量过滤器。随机查看 20 个样例，将过滤器预测与你自己的判断进行比较。说明质量过滤器与人工判断不一致的案例。

交付内容：用 2-5 句话回答。

2.7 质量分类器

接下来，我们将超越 Gopher 规则所捕捉的简单句法质量标准。语言模型训练并不是第一个需要按质量给内容排序的领域；在信息检索中，文本质量同样是一个基础问题。搜索引擎利用的经典信号之一是网页链接结构：高质量网页往往会链接到其他高质量网页[3]。OpenAI 在构建 GPT-2[4] 的训练数据集 WebText 时使用了类似思路：收集那些被 Reddit 评论链接、且评论“karma”分数达到最低阈值的网页。Wikipedia 也可替代 Reddit 作为高质量链接来源，因为 Wikipedia 页面链接到的外部来源通常较为可信[5]。

使用受控来源通常可以获得较高质量的内容，但以今天的标准来看，生成的数据集会很小。OpenWebText 只有 40GB 文本，而 The Pile 的规模超过它的 20 倍。一种方法是将这些参考网页作为正样例，将 Common Crawl 中随机抽取的网页作为负样例，并训练 fastText 分类器。该分类器会输出质量分数，可用于过滤完整的 Common Crawl。设置质量阈值时，需要在精确率与召回率之间权衡。

本部分中，你将构建一个质量分类器。为方便起见，课程组已经从近期 Wikipedia 数据转储中抽取了参考网页 URL，并放置在 /shared-data/wiki/enwiki-20260501-extracted_urls.txt.gz。该文件包含截至 2026 年 5 月英文 Wikipedia 页面中的外部链接。你需要从中抽样，获得用于训练分类器的“高质量”正样例。注意，这些正样例仍可能含有不希望保留的内容，因此可以使用之前构建的原语，例如语言识别与过滤规则，进一步提升它们的质量。

给定一个 URL 文件，可以使用 wget 抓取网页内容，并保存为 WARC 格式：

```
wget --timeout=5 \
 -i subsampled_positive_urls.txt \
 --warc-file=subsampled_positive_urls.warc \
 -O /dev/null
```

题目（quality_classifier）：质量分类器（15 分）

(a) 训练一个质量分类器，使其在给定文本后返回数值型质量分数。

交付内容：一个可用于下一小题的质量分类器。

(b) 编写一个函数，将网页标记为高质量或低质量，并给出该标签的置信度分数。

交付内容：实现一个函数，只接收一个字符串参数，返回由标签（是否高质量）和置信度分数组成的二元组。实现适配器 [run_classify_quality]。作为健全性检查，请运行 uv run pytest -k test_classify_quality，确保能正确分类课程组提供的两个样例。

3 去重

互联网中包含大量重复内容。有些网页彼此完全相同，例如归档页面，或由常见工具生成的默认页面，如常见 Web 服务器的 404 页面。但多数重复发生在更细粒度上。以 Stack Overflow 的问题页面为例：每个页面都有独特内容，例如问题、评论和回答；但所有页面也共享大量重复部分，例如页眉、菜单

选项和页脚，这些内容在各页面渲染时会以完全相同的形式反复出现。本节前半部分处理这种精确重复，随后再讨论近似重复。

3.1 精确行去重

一种简单的精确去重方法，是只保留在整个语料库中唯一出现的行。事实证明，仅用这一方法便足以消除大量冗余，例如前文提到的页眉和菜单选项。在较简单的情况下，删除在其他位置完全重复的行后，页面中往往只剩下独特的主体内容，例如 StackOverflow 的问题与回答。

实现时，可以先遍历语料库一次，统计每一行的出现次数；再进行第二次遍历，重写每个文档，只保留唯一行。

朴素实现中，计数器所用的数据结构可能需要与语料库中全部唯一行相当的存储空间。一个简单的内存优化技巧，是用每一行的哈希值作为键，使键长固定，而不是随行长变化。你现在将实现这种简单去重方法。

题目（exact_deduplication）：精确行去重（3 分）

编写一个函数，输入一组文件路径，并对这些文件执行精确行去重。函数应先使用哈希降低内存占用，统计整个语料库中每一行的频率；随后重写每个文件，仅保留唯一行。

> **交付内容：**实现精确行去重函数。函数接收两个参数：（a）输入文件路径列表；（b）输出目录。它应在输出目录中以相同文件名重写每个输入文件，并删除在全部输入文件中出现超过一次的行。例如，输入路径为 a/1.txt 和 a/2.txt，输出目录为 b/，则函数应写出 b/1.txt 和 b/2.txt。实现适配器 [run_exact_line_deduplication]，并确保通过 uv run pytest -k test_exact_line_deduplication。

3.2 MinHash + LSH 文档去重

精确去重可以删除在多个网页中逐字重复的内容，但无法处理文档内容略有差异的情况。例如软件许可证文档：许可证通常由模板生成，需要填入年份与软件作者姓名。因此，一个采用 MIT 许可证的项目与另一个 MIT 许可证项目的许可证文件大部分内容相同，却并非完全相同。若要移除这种主要由模板生成、仅存在细微差异的重复内容，就需要模糊去重。为高效执行文档级模糊去重，我们将使用 MinHash 与局部敏感哈希（Locality-Sensitive Hashing，LSH）。[5]

模糊去重使用以下文档相似度定义：两个文档 n-gram 集合之间的 Jaccard 相似度。集合 S 与 T 的 Jaccard 相似度定义为：

$$J(S, T) = |S \cap T| / |S \cup T|$$

朴素方法是将每个文档表示为 n-gram 集合，计算所有文档对之间的 Jaccard 相似度，并将超过指定阈值的文档对标记为重复。但对 Common Crawl 这样的大型文档集合而言，该方法不现实；同时，直接存储 n-gram 集合可能比存储文档本身消耗更多内存。

MinHash

为解决内存问题，我们使用签名替代文档 n-gram 集合。目标是构造一种签名，使得比较两个文档的签名时，能够近似得到各自 n-gram 集合之间的 Jaccard 相似度。MinHash 签名具备这一性质。

设文档 n-gram 集合 $S = \{s_1, ..., s_n\}$，计算其 MinHash 签名需要 k 个彼此不同的哈希函数 $h_1, ..., h_k$。[6] 每个哈希函数都将一个 n-gram 映射为整数。对于哈希函数 h_i，集合 S 的 MinHash 定义为：

$$\mathbf{minhash}(h_i, S) = \mathbf{min}(h_i(s_1), h_i(s_2), ..., h_i(s_n))$$

文档 n-gram 集合 S 的签名是 $\mathbb{R}^k$ 中的一个向量，第 i 个元素是 S 在随机哈希函数 h_i 下的 MinHash，即：

$$[\mathbf{minhash}(h_1,S), \mathbf{minhash}(h_2,S), ..., \mathbf{minhash}(h_k,S)]$$

对于两个文档的 n-gram 集合 S_1 和 S_2，它们的 Jaccard 相似度可以用两份签名中 MinHash 值相同的列所占比例近似。证明可参阅文献[6]第 3 章 3.3.3 节。例如，给定文档签名 [1, 2, 3, 2] 和 [5, 2, 3, 4]，第二列和第三列的 MinHash 值相同，因此对应 n-gram 集合的 Jaccard 相似度估计为 2/4。

局部敏感哈希（LSH）

虽然 MinHash 提供了节省内存、并在期望意义上保留任意文档对相似度的文档表示，但我们仍需比较所有文档对，才能找出相似度最高的文档。LSH 提供了一种高效分桶方法，使相似度可能较高的文档落入同一桶中。

对文档签名应用 LSH 时，将 $\mathbb{R}^k$ 中的签名向量划分为 b 个 band，每个 band 含有 r 个 MinHash 值，并满足 $k = br$。例如，若文档签名包含 100 个元素，即由 100 个随机哈希函数生成，可以划分为 2 个 band、每个 50 个 MinHash；也可以划分为 4 个 band、每个 25 个；还可以划分为 50 个 band、每个 2 个。

若两个文档在某个 band 中具有完全相同的哈希值，它们会被聚类到同一个桶中，并被视为候选重复文档。因此，在签名长度固定的情况下，增加 band 数量会提高召回率、降低精确率。

举例而言，设文档 D_1 的 MinHash 签名为 [1, 2, 3, 4, 5, 6]，文档 D_2 的签名为 [1, 2, 3, 5, 1, 2]。若使用 3 个 band、每个 band 含 2 个 MinHash，则 D_1 的三个 band 分别为 [1,2]、[3,4]、[5,6]；D_2 的三个 band 分别为 [1,2]、[3,5]、[1,2]。两份文档的第一个 band 相同，因此会在第一个 band 下被聚到一起。其他 band 不匹配，因此不会在这些 band 下聚到一起。不过，只要至少有一个 band 匹配，它们就会被视为候选重复文档，无论其他 band 是否匹配。

确定候选重复文档后，可以采用多种处理方式。例如，对所有候选重复文档对计算真实 Jaccard 相似度，并把超过指定阈值的文档对标记为重复。

最后，需要跨桶聚类重复文档。例如，文档 A 与 B 在某个桶中匹配，且真实 Jaccard 相似度高于阈值；文档 B 与 C 在另一个桶中匹配，且真实 Jaccard 相似度同样高于阈值。此时应将 A、B、C 视为同一个簇，并在每个簇中随机保留一个文档，删除其余文档。

题目（minhash_deduplication）：MinHash + LSH 文档去重（8 分）

编写一个函数，输入一组文件路径，使用 MinHash 和 LSH 执行模糊文档去重。具体来说，函数应为给定路径中的每个文档计算 MinHash 签名，使用指定数量的 band 执行 LSH，以识别候选重复文档；随后计算候选重复文档之间真实的 n-gram Jaccard 相似度，并移除相似度超过给定阈值的文档。

为提高召回率，请遵循文献[7]的做法，在计算 MinHash 签名及/或比较 Jaccard 相似度之前对文本进行规范化：转为小写、移除标点、规范化空白、去除重音符号，并应用 NFD Unicode 规范化。

> **交付内容：**实现模糊文档去重函数。函数至少应接收以下参数：（a）输入文件路径列表；（b）计算 MinHash 签名所用的哈希函数数量；（c）LSH 的 band 数量；（d）计算 MinHash 签名时采用的词级 n-gram 长度；（e）输出目录。可以假设哈希数量能够被 band 数量整除。
>
> **交付内容：**函数应在输出目录中以相同文件名重写每个输入文件，但只写出以下文档：（a）不是候选重复文档；或（b）从聚类桶中随机选中并保留的文档。例如，输入路径为 a/1.txt 和 a/2.txt，输出目录为 b/，则函数应写出 b/1.txt 和 b/2.txt。实现适配器 [run_minhash_deduplication]，并确保通过 uv run pytest -k test_minhash_deduplication。

[5] 更深入的 LSH 与 MinHash 介绍见文献[6]第 3 章：infolab.stanford.edu/~ullman/mmds/ch3n.pdf。

[6] 这 k 个不同的哈希函数可以来自同一哈希函数族，但使用不同随机种子。例如，MurmurHash3 是一个哈希函数族，指定一个种子即可实例化其中一个具体哈希函数。

4 为语言模型过滤数据

现在，我们已经实现了多种用于过滤网络抓取数据的基础原语，接下来将把它们组合起来，生成语言模型训练数据。

本部分的目标是过滤一组 Common Crawl WET 文件，生成语言模型训练数据。起始数据是已经按英语进行过过滤的 WET 文件：课程组使用 fastText 的 lid.176.bin 语言识别模型，只保留英语概率至少为 70% 的文档。这些英语过滤后的 WET 文件位于 /shared-data/english-wet-data。

具体目标是过滤 CC 数据，使基于过滤后数据训练的 Transformer 语言模型在 Paloma 基准[8]的 C4 100 domains 子集上获得尽可能低的验证困惑度。不得修改模型架构或训练过程，因为本题的目标是构造最优数据。该验证集包含 C4 语言模型数据集[9]中出现频率最高的 100 个域名的样例。课程组已在 /shared-data/tokenized_paloma_c4_100_domains_validation.bin 提供该数据的副本，并使用 GPT-2 tokenizer 完成分词。你可以浏览该数据，以了解其内容。加载方式如下：

```
import numpy as np

data = np.fromfile(
    "/shared-data/tokenized_paloma_c4_100_domains_validation.bin",
    dtype=np.uint16
)

from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("gpt2")
print(tokenizer.decode(data[0:2000]))
```

获得过滤后的数据集后，你将在这些数据上训练一个形状配置类似 GPT-2 small 的模型，并在 C4 100 上评估困惑度。

警告：允许使用 Paloma 验证数据来构建过滤器或分类器，以处理 CC WET 文件；但禁止将任何验证数据原样复制进训练数据。语言模型绝不能看到验证集中的数据。

即便只是这批 WET 文件，数据量也相当可观。为高效处理，建议尽可能使用多进程。Python 的 concurrent.futures 或 multiprocessing API 可能会有帮助。下面给出一个使用 concurrent.futures 在多个进程中并行执行函数的最小示例：

```
import concurrent.futures
import os

from tqdm import tqdm


def process_single_wet_file(input_path: str, output_path: str):
    # TODO: 读取 input_path，处理输入，并将结果写入 output_path
    return output_path


# 创建执行器
num_cpus = len(os.sched_getaffinity(0))
executor = concurrent.futures.ProcessPoolExecutor(max_workers=num_cpus)
wet_filepaths = ["a.warc.wet.gz", "b.warc.wet.gz", "c.warc.wet.gz"]
output_directory_path = "/path/to/output_directory/"
futures = []

for wet_filepath in wet_filepaths:
    # 对每个 warc.wet.gz 文件提交任务，并取得一个 future
    wet_filename = str(pathlib.Path(wet_filepath).name)
    future = executor.submit(
        process_single_wet_file,
        wet_filepath,
        os.path.join(output_directory_path, wet_filename)
    )
    # 保存 future
    futures.append(future)

# 按完成顺序遍历 future，并用进度条追踪进展
for future in tqdm(
    concurrent.futures.as_completed(futures),
    total=len(wet_filepaths),
):
    output_file = future.result()
    print(f"Output file written: {output_file}")
```

对较大的文本块，可使用 concurrent.futures 或 multiprocessing 在本地并行处理。若要在 Modal 的多个 worker 间并行，可对输入调用 .map(...)。

课程组还建议使用 FastWARC 库遍历每个 WET 文件中的记录，并使用 tldextract 库从 URL 中提取域名以进行过滤。以下类可能会有帮助：

```
from fastwarc.warc import ArchiveIterator, WarcRecordType
from tldextract import TLDExtract
```

题目（filter_data）：为语言模型过滤数据（6 分）

(a) 编写脚本，从 /shared-data/english-wet-data 下的一组 Common Crawl WET 文件中筛选语言模型训练数据。你可以自由使用此前实现的任意原语，也可以探索其他过滤器与数据生成方法，例如根据 n-gram 语言模型困惑度进行过滤。目标是生成一份数据，使模型在其上训练后，能够尽量降低 Paloma 基准 C4 100 domains 子集上的困惑度。

警告：再次强调：允许使用 Paloma 验证数据构造过滤器或分类器，以处理 CC WET 文件；但不得将任何验证数据原样复制到训练数据中。

脚本应报告每个过滤步骤保留的样例数量，以便理解各个过滤器对最终输出数据的贡献。

交付内容：一个脚本或一系列脚本：并行过滤课程组提供的 CC WET 文件，并生成语言模型训练数据；同时书面说明每个过滤步骤分别移除了全部丢弃样例中的多大比例。

(b) 过滤课程组提供的 WET 文件（原始共 2,500 个 WET 文件）需要多长时间？若过滤整个 Common Crawl 数据转储，需要多长时间？

交付内容：数据过滤流水线的运行时间。

题目（inspect_filtered_data）：检查过滤后的数据（4 分）

(a) 从过滤后的数据集中随机选取 5 个样例，评价其质量以及是否适合用于语言模型训练，尤其要结合“尽量降低 C4 100 domains 基准困惑度”这一目标。

交付内容：给出最终过滤数据中的 5 个随机样例。文档可能很长，只展示相关片段即可。对每个样例，用 1-2 句话说明其内容，并判断是否值得用于语言模型训练。

(b) 从被过滤脚本删除和/或修改的 CC WET 文档中随机选取 5 个样例。你的过滤流程中的哪个步骤删除或修改了这些文档？你认为删除或修改是否合理？

交付内容：给出原始 WET 中 5 个随机丢弃样例。只展示相关片段即可。对每个样例，用 1-2 句话描述，并判断其删除是否合理。

(c) 若上述分析提示需要进一步修改数据流水线，可以在训练模型之前完成这些修改。请报告你尝试过的任何数据改动与迭代。

交付内容：描述你尝试过的数据修改与迭代。

在训练语言模型之前，需要对数据进行分词。使用 transformers 中的 GPT-2 tokenizer，将过滤后的数据编码为训练所需的整数 ID 序列。不要忘记在每个文档末尾加入 GPT-2 序列结束标记 <|endoftext|>。下面给出起始代码：

```
import multiprocessing
import numpy as np
from tqdm import tqdm
from transformers import AutoTokenizer

input_path = "path/to/your/filtered/data"
output_path = "path/to/your/tokenized/data"

tokenizer = AutoTokenizer.from_pretrained("gpt2")


def tokenize_line_and_add_eos(line):
    return tokenizer.encode(line) + [tokenizer.eos_token_id]


with open(input_path) as f:
    lines = f.readlines()

pool = multiprocessing.Pool(multiprocessing.cpu_count())
chunksize = 100
results = []

for result in tqdm(
    pool.imap(tokenize_line_and_add_eos, lines, chunksize=chunksize),
    total=len(lines),
    desc="Tokenizing lines"
):
    results.append(result)

pool.close()
pool.join()

# 展平 ID 列表并转换为 NumPy 数组
all_ids = [token_id for sublist in results for token_id in sublist]
print(f"Tokenized and encoded {input_path} into {len(all_ids)} tokens")
ids_array = np.array(all_ids, dtype=np.uint16)
ids_array.tofile(output_path)
```

题目（tokenize_data）：数据分词（2 分）

编写脚本，对过滤后的数据进行分词并序列化。务必按照上方示例，使用 ids_array.tofile(output_path) 写出结果，其中 ids_array 是 dtype 为 np.uint16 的 NumPy 整数 ID 数组。这样才能与课程组提供的训练脚本兼容。

你的过滤数据集中共有多少个 token?

交付内容：一个对过滤数据进行分词和序列化的脚本，以及最终数据集中的 token 数量。

完成数据分词后，即可在其上训练模型。我们将训练一个形状配置类似 GPT-2 small、约含 4.3 亿参数的模型，并定期在 C4 100 domains 数据集上测量验证性能。训练使用 8 张 B200 GPU，采用数据并行，每个设备的 batch size 为 128，总优化步数为 16,384。上下文长度为 512，因此训练期间采样的训练 token 数约为 2^{33} = 86 亿。课程组使用该配置完成一次训练约需 2 小时。

使用 scripts/train.py 中提供的 train.py 脚本启动训练。[7] 可在 cs336_basics/train_config.py 中查看要使用的超参数。通过 --train-bin 传入分词后训练数据的路径：

```
uv run modal run scripts/train.py --train-bin /root/data/your_data.bin
```

再次强调，本次作业的目标是通过优化数据降低验证损失，而不是修改模型和/或优化过程来降低损失，因此不要修改训练流程或训练脚本。

随后，可以使用以下命令从保存的模型中生成样例：

```
uv run modal run scripts/generate_with_gpt2_tok.py --model-path /root/data/output/your_data
```

题目（train_model）：训练模型（8 分）

在分词后的数据集上训练一个语言模型（形状配置类似 GPT-2 small，约 4.3 亿参数）。定期测量其在 C4 100 domains 上的验证损失。模型达到的最佳验证损失是多少？

交付内容：记录到的最佳验证损失、对应的学习曲线，以及对你所做工作的描述。

[7] https://github.com/stanford-cs336/assignment4-data/blob/master/scripts/train.py

参考文献

[1] L. Soldaini et al., “Dolma: An Open Corpus of Three Trillion Tokens for Language Model Pretraining Research,” arXiv preprint arXiv:2402.00159, 2024.

[2] J. W. Rae et al., “Scaling language models: Methods, analysis & insights from training gopher,” arXiv preprint arXiv:2112.11446, 2021.

[3] L. Page, S. Brin, R. Motwani, and T. Winograd, “The pagerank citation ranking: Bring order to the web,” in Proc. of the 7th International World Wide Web Conf, 1998.

[4] A. Radford et al., “Language models are unsupervised multitask learners,” OpenAI blog, vol. 1, no. 8, p. 9, 2019.

[5] H. Touvron et al., “LLaMA: Open and Efficient Foundation Language Models.” 2023.

[6] J. Leskovec, A. Rajaraman, and J. D. Ullman, Mining of Massive Datasets, 2nd ed. USA: Cambridge University Press, 2014.

[7] G. Penedo et al., “The RefinedWeb Dataset for Falcon LLM: Outperforming Curated Corpora with Web Data, and Web Data Only.” 2023.

[8] I. Magnusson et al., “Paloma: A Benchmark for Evaluating Language Model Fit.” 2023.

[9] C. Raffel et al., “Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer,” Journal of Machine Learning Research, vol. 21, no. 140, pp. 1-67, 2020, [Online]. Available: http://jmlr.org/papers/v21/20-074.html